Note technique

Architecture

Pour ce projet, nous avons décidé de conserver l'architecture de base fournie par notre intervenant.

Le projet était déjà séparé correctement en plusieurs fichiers, ce qui nous permettait de travailler sur différentes fonctionnalités sans avoir à modifier l'ensemble du programme.

Le fichier `main.cpp` s'occupe principalement de l'affichage du jeu en 3D avec raylib, des commandes du joueur et des informations affichées à l'écran (HUD).

Le fichier `simulation.cpp` gère la mise à jour générale de la simulation et la logique des objets (update) , notamment les déplacements du joueur, les ennemis et les interactions avec le gameplay.

Nous avons également utilisé des fichiers spécifiques pour les différentes fonctionnalités, comme `pathfinding.cpp` pour les algorithmes de recherche de chemin, `navmesh.cpp` pour la navigation, `navigation_agent.cpp` pour le déplacement des ennemis et `enemy_ai.cpp` pour leurs comportements.

Nous avons organisé le jeu autour de trois vagues d'ennemis, chacune associée à une carte différente : `arena.level`, `pillars.level` et `tricky.level`.Quand la wave augmente, on recharge une nouvelle map.

Nous avons rencontré un problème à ce moment-là, car le constructeur de `Simulation` appelle `Reset()`, ce qui remet normalement la progression à zéro. Pour éviter cela,on a dû sauvegarder la vie du joueur (`player.hp`) et la vague (`currentWave`) pour les remettre dans la nouvelle simulation.

Ce choix a été fait pour garder l'architecture de base sans devoir tout reorganiser.

Pour la gestion de mémoire, Nous utilisons `std::unique_ptr` pour supprimer proprement l'ancienne instance de niveau avant de charger la nouvelle map.

Coûts et heuristique

Nous avons implémenté deux algorithmes de recherche de chemin : Dijkstra et A*.

Ils permettent de trouver un trajet entre deux polygones du NavMesh, en utilisant un graphe dans lequel les polygones représentent les nœuds et les connexions entre eux représentent les arêtes.

Dijkstra fonctionne en recherchant progressivement les chemins ayant le coût le plus faible depuis le point de départ.

A* utilise le même principe, mais ajoute une heuristique pour estimer la distance restante jusqu'à la destination.

Sa priorité est calculée avec la formule `f(n) = g(n) + h(n)`, où `g(n)` représente le coût déjà parcouru et `h(n)` le coût restant estimé.

Pour notre heuristique, nous avons utilisé la distance euclidienne entre le centre du polygone actuel et celui du polygone de destination. Ce choix était adapté à notre NavMesh puisque nous calculions déjà les centres des polygones lors de sa construction. Il permet à A* d'orienter sa recherche vers l'objectif, contrairement à Dijkstra qui ne prend en compte que le coût parcouru.

Concernant le calcul des coûts des arêtes, nous utilisons le milieu du portail qui relie deux polygones, calculé avec la formule `middle = (p.p + p.q) * 0.5`.

Le coût correspond donc à la distance entre le centre du premier polygone et le milieu du portail, à laquelle on ajoute la distance entre ce milieu et le centre du deuxième polygone :

```
double cost = Distance(polygons_[p.a].center, middle) +
Distance(middle, polygons_[p.b].center);
```

Cela permet d'avoir des coûts liés à la géométrie du niveau plutôt que de donner la même valeur à toutes les connexions, tout en favorisant des trajectoires qui passent par le milieu des portails.

Avec ce calcul, notre heuristique ne surestime pas le coût restant sur le graphe.

A* peut donc trouver un chemin de coût minimal dans ce graphe. En revanche, le chemin final n'est pas forcément le trajet le plus court possible dans le niveau, car nous utilisons les centres des polygones et les milieux des portails comme points de passage.

Les deux algorithmes utilisent une file de priorité et enregistrent les prédécesseurs des nœuds pour reconstruire le chemin une fois la destination atteinte.

NavMesh et rayon

Le NavMesh est construit à partir des polygones présents dans les fichiers de niveaux. Nous calculons le centre de chaque polygone, puis nous recherchons les arêtes communes pour créer les portails qui relient les différentes zones.

Le rayon des agents (`agentRadius`) permet de prendre en compte leur taille lors des déplacements et des collisions, afin d'éviter qu'ils traversent les murs ou passent trop près des obstacles.

Il est également utilisé pour vérifier qu'un agent dispose de suffisamment de place pour traverser un portail. Dans notre code, nous comparons la largeur du portail au diamètre de l'agent, avec une marge de sécurité de 0,05. Cela permet de refuser un chemin lorsqu'un passage est trop étroit. Cependant, cette vérification se fait après le calcul du chemin. Si un portail est trop étroit, le programme ne recherche pas automatiquement un autre itinéraire.

Nous avons également ajouté une gestion des collisions entre ennemis dans `Simulation::Update()`. Pour cela, nous utilisons une double boucle qui compare la position des ennemis deux par deux.

Si la distance entre deux ennemis vivants est inférieure à `2 * level.agentRadius`, nous calculons leur chevauchement et les décalons légèrement dans des directions opposées.

Cela permet d'éviter que plusieurs ennemis se superposent lorsqu'ils poursuivent le joueur.

Automate

Pour gérer l'intelligence artificielle des ennemis, nous avons utilisé un automate à états. Cela permet de séparer leurs différents comportements et de définir les conditions qui les font passer d'un état à un autre.

Notre automate comporte cinq états : `Idle`, `Chase`, `Windup`, `Recover` et `Dead`.

Dans l'état `Idle`, l'ennemi attend que le joueur entre dans sa zone de détection. Il passe alors à `Chase`, où il utilise le système de navigation pour se rapprocher de lui. Lorsqu'il est suffisamment proche et qu'un segment direct jusqu'au joueur reste dans le NavMesh, il passe à `Windup`, qui correspond à la préparation de son attaque.

Une fois le temps de préparation terminé, l'ennemi vérifie à nouveau sa distance avec le joueur avant d'infliger des dégâts.

Il passe ensuite à `Recover`, pendant lequel il doit attendre avant de pouvoir reprendre la poursuite.

Enfin, lorsque ses points de vie atteignent zéro, il passe à `Dead` et sa navigation est arrêtée. Si le joueur meurt ou s'éloigne au-delà de la distance maximale, l'ennemi revient à `Idle`.

Nous avons deux types d'ennemis : les Light et les Heavy.

Les Light sont plus rapides, possèdent 20 points de vie et infligent 8 dégâts. Cependant, ils fuient quand le joueur approche trop près.

Les Heavy sont plus lents, mais possèdent 80 points de vie et infligent 25 dégâts. Leur attaque demande aussi davantage de temps de préparation.

Pour le combat du joueur, nous avons ajouté une attaque directionnelle. Au départ, l'attaque pouvait toucher tous les ennemis proches, même ceux situés derrière le personnage.

Nous avons donc utilisé le produit scalaire entre la direction du joueur et celle de l'ennemi pour vérifier qu'il se trouve suffisamment devant lui. L'attaque inflige 10 dégâts aux ennemis concernés lorsqu'ils sont à moins de 1,5 unité.

Le jeu comporte trois vagues de difficulté progressive, avec 3, puis 6, puis 10 ennemis.

À partir de la deuxième vague, des Heavy peuvent apparaître aléatoirement. Les ennemis sont générés à des positions aléatoires vérifiées par rapport au NavMesh et aux collisions physiques. Lorsque tous les ennemis d'une vague sont morts, la progression passe à la suivante. Après la troisième vague, le jeu affiche la victoire.

Mesures

Pour comparer Dijkstra et A*, nous avons prévu deux indicateurs dans notre programme :

le nombre de nœuds développés et le temps nécessaire pour calculer un chemin.

Le nombre de nœuds développés est enregistré dans `result.expanded`, tandis que le temps d'exécution est mesuré en microsecondes avec `std::chrono::steady_clock`. Ces informations peuvent être affichées dans le HUD lorsqu'une requête de chemin est effectuée avec la souris.

Pour comparer correctement les deux algorithmes, nous devons utiliser les mêmes positions de départ et d'arrivée. Cela nous permettra de voir si A* explore moins de nœuds que Dijkstra et de comparer leurs temps d'exécution.

- Ajout de timers (`playerFlashTimer` et `enemyFlashTimers`).

- Quand un perso perd des PV, son timer passe a 0.2s et son model passe en rouge temporairement pour avertir le joueur.

Anomalies et choix de périmètre

Pendant le développement, nous avons rencontré plusieurs problèmes liés à l'intégration du gameplay.

-Le premier concernait le son des attaques ennemies, qui se jouait à chaque frame pendant toute la durée de l'état `Windup`.

Pour corriger cela, nous avons ajouté `lastEnemyStates` dans `main.cpp` pour jouer PlaySound uniquement lors de la frame ou l'état passe à `Windup`

-Nous avons également rencontré un problème avec l'attaque au corps à corps du joueur. Au départ, elle infligeait des dégâts à tous les ennemis proches, même ceux situés derrière lui. Pour corriger cela, nous avons utilisé le produit scalaire `Dot(player.facing, toEnemy)`. Si le résultat est supérieur à `0.4`, l'ennemi est considéré comme étant suffisamment devant le joueur pour recevoir des dégâts. Sinon, il n'est pas touché.

-Un autre problème concernait le changement de map. Lorsque nous passions à la deuxième map, la simulation se réinitialisait à la première vague, avec seulement trois ennemis.

Pour résoudre ce problème, nous avons conservé le numéro de vague en ajoutant `next->currentWave = sim->currentWave` après la création de la nouvelle simulation avec `std::make_unique`.

Nous avons aussi ajouté des retours visuels et sonores pour rendre les combats plus compréhensibles. Lorsqu'un personnage reçoit des dégâts, il apparaît brièvement en rouge pendant 0,2 seconde. Des sons accompagnent également certaines actions, notamment les attaques et la mort des ennemis.

Concernant le périmètre du projet, nous avons réalisé trois maps qui s'enchaînent : `arena.level`, `pillars.level` et `tricky.level`.

Nous avons intégré un système de combat au corps à corps qui prend en compte la direction du joueur, ainsi qu'une IA composée de cinq états et de deux types d'ennemis, Light et Heavy.

Nous avons cependant choisi de ne pas ajouter d'assets graphiques personnalisés. Le jeu utilise principalement les formes géométriques de base de raylib, sans textures avancées ni décors complexes. Nous n'avons pas non plus développé de comportement d'intelligence artificielle de groupe complexe : chaque ennemi prend ses décisions individuellement.

Nous avons préféré nous concentrer sur le bon fonctionnement des principales fonctionnalités du projet, notamment la navigation, les algorithmes de recherche de chemin, le gameplay et le comportement des ennemis.